

**ECOLE NORMALE SUPÉRIEURE DE
L'ENSEIGNEMENT TECHNIQUE DE
MOHAMMEDIA**

UNIVERSITÉ HASSAN II DE CASABLANCA

Compte rendu du mini projet

WatchTower

Multi-Server Remote Monitor

Module : Théorie des systèmes d'exploitation & SE Windows/Unix/Linux

Réalisé par :

Mohamed Amine El haouari
Jaffal Mohammed
Aissam Bamouh
Touil Saif Yassine

Professeur encadrant :

Abdellah OUAGUID

Année universitaire : 2025–2026

Date : 14/05/2026

Table des matières

1	Introduction	2
2	Contexte et besoin identifié	2
3	Objectif du projet	2
4	Architecture générale du projet	3
5	Options implémentées	3
6	Notions shell et système exploitées	3
7	Journalisation	4
8	Gestion d’erreurs	4
9	Principe de fonctionnement	4
9.1	Lecture du fichier de configuration	4
9.2	Collecte des métriques	5
9.3	Évaluation des états	5
10	Modes d’exécution	5
10.1	Mode subshell	5
10.2	Mode fork	5
10.3	Mode thread	5
11	Scénarios de test	5
12	Captures d’écran et commentaires	6
12.1	Aide et informations générales	6
12.2	Préparation et démonstration initiale	6
12.3	Tableaux de bord et supervision	7
12.4	Scénario léger, moyen et lourd	8
12.5	Illustration des modes fork et thread	8
12.6	Journalisation et maintenance	9
12.7	Seuils personnalisés et alertes	10
13	Analyse des résultats	11
14	Conclusion	12

1 Introduction

Ce document constitue le compte rendu détaillé du mini projet demandé dans l'énoncé `Devoir_Mini_Projet_v1.0.pdf`. Le travail réalisé porte sur le développement d'un script Bash nommé `watchtower`, destiné à superviser plusieurs serveurs à partir d'une seule interface terminale.

L'objectif global est d'automatiser la collecte d'informations système, la détection d'anomalies, la journalisation des événements et l'émission d'alertes, tout en respectant les directives imposées dans le sujet : paramètre obligatoire, options multiples, exécution en `subshell`, `fork` et `thread`, gestion d'erreurs, contrôle d'accès et documentation intégrée.

2 Contexte et besoin identifié

Dans un environnement d'administration moderne, la surveillance des serveurs est une tâche répétitive, sensible aux erreurs humaines et coûteuse en temps. Lorsqu'un administrateur doit suivre plusieurs machines Linux, Windows et Docker, il lui faut vérifier régulièrement :

- l'utilisation du processeur ;
- le taux d'occupation mémoire ;
- l'espace disque utilisé ;
- la disponibilité de certains services ;
- la connectivité réseau ;
- les erreurs d'authentification ou indisponibilités.

La réalisation manuelle de ces vérifications ne permet ni une bonne centralisation ni une réaction rapide en cas d'incident. Le besoin retenu consiste donc à concevoir un outil unique, simple à lancer, capable de superviser plusieurs cibles et de produire des traces exploitables.

3 Objectif du projet

Le script `watchtower` répond à ce besoin en mettant en place les fonctionnalités suivantes :

- lecture d'un fichier de configuration contenant plusieurs serveurs ;
- collecte automatique des métriques CPU, RAM, disque, latence et services ;
- affichage d'un tableau de bord actualisé en continu ;
- stockage des informations et des erreurs dans un journal `history.log` ;
- prise en charge de plusieurs stratégies d'exécution ;
- envoi d'alertes Telegram en cas d'état critique ou d'indisponibilité.

La syntaxe générale du programme respecte la forme d'une commande Linux standard :

```
./watchtower [options] -c servers.conf
```

4 Architecture générale du projet

Le projet s'appuie sur plusieurs fichiers complémentaires :

Fichier	Rôle
<code>watchtower</code>	script Bash principal qui orchestre la lecture de configuration, la collecte, l'affichage, les alertes et la journalisation ;
<code>watchtower_threads.c</code>	programme C utilisant <code>pthread</code> pour matérialiser le mode thread ;
<code>run-watchtower-demo.sh</code>	script de démonstration et de préparation de l'environnement ;
<code>servers.conf</code>	scénario de test lourd ;
<code>servers-medium.conf</code>	scénario de test moyen ;
<code>servers-light.conf</code>	scénario de test léger ;
<code>telegram.local.example</code>	
<code>.env</code>	exemple de configuration pour Telegram.

5 Options implémentées

Le programme contient les options demandées dans l'énoncé, ainsi que quelques extensions utiles :

Option	Description
<code>-h</code>	affiche l'aide détaillée du programme ;
<code>-c <file></code>	paramètre obligatoire permettant de préciser le fichier de configuration ;
<code>-s</code>	exécution en mode <code>subshell</code> ;
<code>-f</code>	exécution en mode <code>fork</code> avec processus fils ;
<code>-t</code>	exécution en mode <code>thread</code> avec helper <code>pthread</code> ;
<code>-l <dir></code>	permet de choisir le répertoire de journalisation ;
<code>-r</code>	réinitialise les journaux, le cache et les baselines ; nécessite les privilèges administrateur ;
<code>-b</code>	archive <code>history.log</code> au format <code>tar.gz</code> ;
<code>-m</code>	active l'envoi d'alertes Telegram ;
<code>-i <seconds></code>	définit l'intervalle de rafraîchissement ;
<code>-a <values></code>	personnalise les seuils CPU, RAM et disque ;
<code>-v</code>	affiche la version du programme.

6 Notions shell et système exploitées

Le projet met en oeuvre plusieurs notions étudiées dans le module :

— variables d'environnement et variables Bash ;

- tableaux, fonctions, conditions et boucles ;
- expressions régulières pour la validation des entrées ;
- redirections, pipes et filtres avec **awk**, **grep**, **sed**, **tee** ;
- manipulation de fichiers, archivage et compression ;
- contrôle d'accès avec vérification de EUID ;
- exécution concurrente en **subshell**, **fork** et **thread** ;
- appel à un programme externe en langage C.

7 Journalisation

Le programme respecte la structure de journalisation demandée par l'énoncé. Les messages standards et les erreurs sont envoyés simultanément au terminal et dans le fichier `history.log`. Le format adopté est :

```
yyyy-mm-dd-hh-mm-ss : username : INFOS : message
yyyy-mm-dd-hh-mm-ss : username : ERROR : message
```

Le dossier par défaut est `/var/log/watchtower`. Si ce répertoire n'est pas accessible en écriture, le script bascule automatiquement vers `./logs`. Cette logique permet de maintenir l'exécution même dans un environnement utilisateur non privilégié.

8 Gestion d'erreurs

Le script associe des codes précis aux erreurs détectées :

- 100 : option inconnue ;
- 101 : paramètre obligatoire manquant ;
- 102 : fichier de configuration introuvable ;
- 103 : format de configuration ou valeur invalide ;
- 104 : échec de connexion SSH ;
- 105 : échec d'une collecte Docker ;
- 106 : erreur de permission ou besoin de droits administrateur ;
- 107 : dépendance manquante.

Après une erreur, le programme réaffiche également la documentation d'aide afin de guider l'utilisateur.

9 Principe de fonctionnement

9.1 Lecture du fichier de configuration

Chaque ligne de configuration décrit un serveur avec les champs suivants :

```
alias host user port os_type services [password]
```

Cette organisation permet de traiter aussi bien des serveurs Linux que des serveurs Windows ou des serveurs de test Docker.

9.2 Collecte des métriques

Trois méthodes principales sont utilisées :

- pour Docker : utilisation de `docker stats` et `docker exec` ;
- pour Linux : connexion SSH puis lecture des fichiers système et des services ;
- pour Windows : exécution de commandes PowerShell via SSH.

Les métriques collectées incluent le CPU, la RAM, le disque, le ping, l'état des services, l'uptime, ainsi que certains éléments de sécurité comme le nombre d'échecs d'authentification.

9.3 Évaluation des états

Chaque serveur reçoit ensuite un état global :

- **HEALTHY** : fonctionnement normal ;
- **WARNING** : service partiellement indisponible ;
- **CRITICAL** : seuil dépassé ;
- **DOWN** : collecte impossible ou serveur injoignable.

10 Modes d'exécution

10.1 Mode subshell

Le mode `subshell` lance la collecte de manière séquentielle à l'intérieur d'un sous-shell. Ce mode est le plus simple à comprendre et sert de référence de base.

10.2 Mode fork

Le mode `fork` crée un processus fils par serveur à l'aide de l'opérateur `&` puis synchronise les résultats avec `wait`. Ce mode est adapté aux scénarios moyens et lourds.

10.3 Mode thread

Le mode `thread` compile et exécute un helper C basé sur `pthread`. Le script enregistre les traces générées par ces threads, puis s'appuie sur les workers Bash pour finaliser la collecte.

11 Scénarios de test

Conformément à l'énoncé, plusieurs scénarios ont été exécutés :

- scénario léger : `servers-light.conf` avec 2 serveurs ;
- scénario moyen : `servers-medium.conf` avec 3 serveurs ;
- scénario lourd : `servers.conf` ou `servers.local.conf` avec 4 à 6 serveurs ;
- tests complémentaires des options `-b`, `-r`, `-m`, `-a` et `-v`.

12 Captures d'écran et commentaires

12.1 Aide et informations générales

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -h
NAME
  watchtower - Multi-Server Remote Health Monitor

SYNOPSIS
  ./watchtower [options] -c servers.conf

DESCRIPTION
  WatchTower monitors Linux servers, Docker test servers, and Windows servers
  from one terminal. It reads a config file, collects CPU/RAM/disk/service
  metrics, displays a live dashboard, and writes events to history.log.

CONFIG FORMAT
  alias host user port os_type services [password]

  Docker test server:
    server1 localhost test 2221 linux sshd

  Windows server example:
    server5 192.168.X.X windows_user 22 windows sshd

  Password example:
    server6 192.168.X.X windows_user 22 windows sshd MyPassword123

OPTIONS
  -h          Show this help
  -c <file>   Required config file for monitoring
  -s          Subshell mode: sequential collection inside ( ... )
  -f          Fork mode: one child process per server, using & and wait
  -t          Thread mode: calls a small C pthread helper, then Bash workers
  -l <dir>     Log directory, default /var/log/watchtower
  -r          Restore/reset logs and cache, admin only
  -b          Backup/compress history.log as tar.gz
  -m          Enable Telegram alerts for CRITICAL and DOWN states
  -i <seconds> Refresh interval, default 5
  -g <values>  Thresholds, example: cpu=80,ram=85,disk=90
  -v          Show version

LOG FORMAT
  yyyy-mm-dd-hh-mm-ss : username : INFOS : message
  yyyy-mm-dd-hh-mm-ss : username : ERROR : message

ERROR CODES
  100 unknown option
  101 missing mandatory config
  102 config file not found
  103 invalid config format
  104 SSH failure
  105 Docker failure
  106 permission/admin error
  107 dependency missing
```

FIGURE 1 – Affichage de l'aide complète via l'option `-h`. Cette capture montre la description du programme, la syntaxe, les options et les codes d'erreur.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -v
[
watchtower v1.0
```

FIGURE 2 – Affichage de la version du programme avec l'option `-v`.

12.2 Préparation et démonstration initiale

```
[jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./run-watchtower-demo.sh
Key installed in server1
Key installed in server2
Key installed in server3
Key installed in server4
Using SSH key: /Users/jaffa/.ssh/watchtower_key
Using Bash: /opt/homebrew/bin/bash
Using config: servers.local.conf
2026-05-14-17-05-01 : jaffa : INFOS : Config loaded: 6 server(s)
2026-05-14-17-05-01 : jaffa : INFOS : WatchTower started in thread mode
./watchtower: line 351: printf: 1.050: invalid number
./watchtower: line 351: printf: 1.327: invalid number
```

FIGURE 3 – Exécution du script `run-watchtower-demo.sh`. Cette étape prépare les clés SSH, sélectionne Bash 4+ et lance une démonstration en mode thread.

12.3 Tableaux de bord et supervision

```
WATCHTOWER v1.0 - Live Multi-Server Monitor [2026-05-14 17:05:23]
```

Server	OS	CPU	RAM	DISK	PING	FAIL	SVC	Status
server1	linux	0%	0%	1%	local	0	1/1	HEALTHY
server2	linux	0%	0%	1%	local	0	1/1	HEALTHY
server3	linux	0%	0%	1%	local	0	1/1	HEALTHY
server4	linux	0%	0%	1%	local	0	1/1	HEALTHY
server5	windows	17%	64.79%	27.68%	0ms	0	1/1	HEALTHY
server6	linux	0%	27%	62%	0ms	22	1/1	HEALTHY

```
Mode: thread | Interval: 2s | Log: /tmp/watchtower_logs/history.log | Ctrl+C to stop
./watchtower: line 351: printf: 0.441: invalid number
./watchtower: line 351: printf: 0.748: invalid number
```

FIGURE 4 – Tableau de bord WatchTower en mode thread sur un scénario lourd. Les six serveurs supervisés sont affichés avec CPU, RAM, disque, ping, nombre d'échecs et état global.

```
[jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main) [101]> ./watchtower -c servers.local.conf
2026-05-14-17-06-57 : jaffa : INFOS : Using ./logs because /var/log/watchtower is not writable
2026-05-14-17-06-57 : jaffa : INFOS : Config loaded: 6 server(s)
2026-05-14-17-06-57 : jaffa : INFOS : WatchTower started in subshell mode
2026-05-14-17-07-06 : jaffa : INFOS : Subshell mode completed
```

```
WATCHTOWER v1.0 - Live Multi-Server Monitor [2026-05-14 17:07:06]
```

Server	OS	CPU	RAM	DISK	PING	FAIL	SVC	Status
server1	linux	0%	0%	1%	local	0	1/1	HEALTHY
server2	linux	0%	0%	1%	local	0	1/1	HEALTHY
server3	linux	0%	0%	1%	local	0	1/1	HEALTHY
server4	linux	0%	0%	1%	local	0	1/1	HEALTHY
server5	windows	ERR	ERR	ERR	1ms	0	N/A	DOWN
server6	linux	0%	26%	62%	1ms	22	1/1	HEALTHY

```
ALERTS:
- server5 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
Mode: subshell | Interval: 5s | Log: ./logs/history.log | Ctrl+C to stop
```

FIGURE 5 – Exécution en mode subshell avec serveur Windows indisponible. Le programme détecte correctement l'état DOWN et ajoute l'alerte correspondante.

```
WATCHTOWER v1.0 - Live Multi-Server Monitor [2026-05-14 17:34:03]
```

Server	OS	CPU	RAM	DISK	PING	FAIL	SVC	Status
server1	linux	0%	0%	1%	local	0	1/1	HEALTHY
server2	linux	100%	0%	1%	local	0	1/1	CRITICAL
server3	linux	0%	0%	1%	local	0	1/1	HEALTHY
server4	linux	0%	0%	1%	local	0	1/1	HEALTHY
server5	windows	ERR	ERR	ERR	0ms	0	N/A	DOWN
server6	linux	8%	35%	63%	0ms	24	1/1	HEALTHY

FIGURE 6 – Exemple de tableau de bord faisant apparaître simultanément un serveur en état CRITICAL et un serveur en état DOWN.

WATCHTOWER v1.0 - Live Multi-Server Monitor [2026-05-14 17:35:50]								
Server	OS	CPU	RAM	DISK	PING	FAIL	SVC	Status
server1	linux	0%	0%	1%	local	0	1/1	HEALTHY
server2	linux	0%	0%	1%	local	0	1/1	HEALTHY
server3	linux	0%	0%	1%	local	0	1/1	HEALTHY
server4	linux	0%	0%	1%	local	0	1/1	HEALTHY
server5	windows	ERR	ERR	ERR	1ms	0	N/A	DOWN
server6	linux	63%	36%	69%	1ms	24	1/1	CRITICAL

ALERTS:

- server5 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
- server6 status=CRITICAL cpu=63 ram=36 disk=69 services=1/1 error=

Mode: thread | Interval: 2s | Log: ./logs/history.log | Ctrl+C to stop

FIGURE 7 – Résultat obtenu après personnalisation des seuils. Le programme signale un serveur **CRITICAL** et un autre **DOWN**.

12.4 Scénario léger, moyen et lourd

```
jajffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -s -i 2 -l ./logs -c servers-light.conf
2026-05-14-17-16-29 : jajffa : INFOS : Config loaded: 2 server(s)
2026-05-14-17-16-29 : jajffa : INFOS : WatchTower started in subshell mode
2026-05-14-17-16-32 : jajffa : INFOS : Subshell mode completed
```

FIGURE 8 – Lancement d'un scénario léger avec `servers-light.conf` en mode subshell.

```
jajffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -t -i 2 -l ./logs -c servers-medium.conf
2026-05-14-17-27-15 : jajffa : INFOS : Config loaded: 3 server(s)
2026-05-14-17-27-15 : jajffa : INFOS : WatchTower started in thread mode
2026-05-14-17-27-17 : jajffa : INFOS : Fork mode completed with 3 child processes
2026-05-14-17-27-17 : jajffa : INFOS : Thread mode used pthread helper and Bash workers
```

FIGURE 9 – Lancement d'un scénario moyen avec `servers-medium.conf` en mode thread. Le log confirme l'utilisation du helper `pthread` et des workers `Bash`.

```
jajffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -t -i 2 -l ./logs -c servers.conf
2026-05-14-17-30-43 : jajffa : INFOS : Config loaded: 4 server(s)
2026-05-14-17-30-43 : jajffa : INFOS : WatchTower started in thread mode
2026-05-14-17-30-44 : jajffa : INFOS : Fork mode completed with 4 child processes
2026-05-14-17-30-44 : jajffa : INFOS : Thread mode used pthread helper and Bash workers
```

FIGURE 10 – Lancement d'un scénario lourd avec `servers.conf` en mode thread.

12.5 Illustration des modes fork et thread

```
jajffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -f -i 2 -l ./logs -c servers.local.conf
2026-05-14-17-23-47 : jajffa : INFOS : Config loaded: 6 server(s)
2026-05-14-17-23-47 : jajffa : INFOS : WatchTower started in fork mode
```

FIGURE 11 – Démarrage du mode `fork` sur une configuration plus lourde.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ps -ef | grep watchtower
501 28835      1  0  5:17PM  ??          0:07.31 /Applications/Visual Studio Code.app/Contents/MacOS/Code --goto /Users/jaffa/watchtower-test
501 41953 13009    0  5:23PM  ttys008    0:00.04 bash ./watchtower -f -i 2 -l ./logs -c servers.local.conf
501 42312 41953    0  5:23PM  ttys008    0:00.00 bash ./watchtower -f -i 2 -l ./logs -c servers.local.conf
501 42345 42312    0  5:23PM  ttys008    0:00.00 bash ./watchtower -f -i 2 -l ./logs -c servers.local.conf
501 42349 42345    0  5:23PM  ttys008    0:00.00 bash ./watchtower -f -i 2 -l ./logs -c servers.local.conf
501 42411 23816    0  5:23PM  ttys009    0:00.00 grep --color=auto watchtower
```

FIGURE 12 – Vérification avec `ps -ef` de la création de plusieurs processus fils lors de l'exécution en mode `fork`.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ls -l watchtower_threads.c
-rw-r--r--@ 1 jaffa  staff  1234 Apr 24 11:10 watchtower_threads.c
```

FIGURE 13 – Présence du fichier `watchtower_threads.c`, utilisé comme helper C pour le mode `thread`.

12.6 Journalisation et maintenance

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> cat ./logs/history.log
2026-04-24-11-11-02 : jaffa : ERROR : Error 100: Unknown option: -z
2026-04-24-11-11-02 : jaffa : INFOS : Backup created: ./logs/watchtower-logs-2026-04-24-11-11-02.tar.gz
2026-04-24-11-11-10 : jaffa : INFOS : Config loaded: 5 server(s)
2026-04-24-11-11-10 : jaffa : INFOS : WatchTower started in thread mode
pthread worker 1 created and joined
pthread worker 2 created and joined
pthread worker 3 created and joined
pthread worker 4 created and joined
pthread worker 5 created and joined
2026-04-24-11-11-16 : jaffa : INFOS : Fork mode completed with 5 child processes
2026-04-24-11-11-16 : jaffa : INFOS : Thread mode used pthread helper and Bash workers
2026-04-24-11-11-16 : jaffa : ERROR : server1 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-16 : jaffa : ERROR : server2 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-16 : jaffa : ERROR : server3 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-16 : jaffa : ERROR : server4 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-16 : jaffa : ERROR : server5 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-16 : jaffa : INFOS : Dashboard refreshed
pthread worker 1 created and joined
pthread worker 3 created and joined
pthread worker 2 created and joined
pthread worker 4 created and joined
pthread worker 5 created and joined
2026-04-24-11-11-24 : jaffa : INFOS : Fork mode completed with 5 child processes
2026-04-24-11-11-24 : jaffa : INFOS : Thread mode used pthread helper and Bash workers
2026-04-24-11-11-24 : jaffa : ERROR : server1 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-24 : jaffa : ERROR : server2 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-24 : jaffa : ERROR : server3 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-24 : jaffa : ERROR : server4 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-24 : jaffa : ERROR : server5 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-11-24 : jaffa : INFOS : Dashboard refreshed
pthread worker 1 created and joined
pthread worker 2 created and joined
pthread worker 3 created and joined
pthread worker 4 created and joined
pthread worker 5 created and joined
2026-04-24-11-11-29 : jaffa : INFOS : WatchTower stopped
2026-04-24-11-12-02 : jaffa : INFOS : Config loaded: 5 server(s)
2026-04-24-11-12-02 : jaffa : INFOS : WatchTower started in thread mode
pthread worker 1 created and joined
pthread worker 2 created and joined
pthread worker 3 created and joined
pthread worker 4 created and joined
pthread worker 5 created and joined
2026-04-24-11-12-08 : jaffa : INFOS : Fork mode completed with 5 child processes
2026-04-24-11-12-08 : jaffa : INFOS : Thread mode used pthread helper and Bash workers
2026-04-24-11-12-08 : jaffa : ERROR : server1 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-12-08 : jaffa : ERROR : server2 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
2026-04-24-11-12-08 : jaffa : ERROR : server3 status=DOWN cpu=ERR ram=ERR disk=ERR services=N/A error=SSH_FAILED
```

FIGURE 14 – Affichage du contenu de `history.log`. On y retrouve les événements d'information, les workers `pthread`, les erreurs et les rafraîchissements du tableau de bord.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> sudo ./watchtower -r -l ./logs -c servers.conf
Password:
2026-05-14-17-31-35 : root : INFOS : Restore completed: logs, cache, and baselines reset.
```

FIGURE 15 – Test de l'option `-r` exécutée avec `sudo`. Le programme réinitialise correctement les logs, le cache et les baselines.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -b -l ./logs -c servers.conf
[
2026-05-14-17-32-39 : jaffa : INFOS : Backup created: ./logs/watchtower-logs-2026-05-14-17-32-39.tar.gz
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ls -lh ./logs
[
total 24
-rw-r--r--@ 1 jaffa  staff   194B May 14 17:32 history.log
-rw-r--r--@ 1 jaffa  staff   398B Apr 24 11:11 watchtower-logs-2026-04-24-11-11-02.tar.gz
-rw-r--r--@ 1 jaffa  staff   418B May 14 17:32 watchtower-logs-2026-05-14-17-32-39.tar.gz
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> █
```

FIGURE 16 – Test de l'option `-b`. Une archive compressée du journal est créée dans le dossier des logs.

12.7 Seuils personnalisés et alertes

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -m -t -i 2 -l ./logs -c servers.local.conf
2026-05-14-17-34-34 : jaffa : INFOS : Config loaded: 6 server(s)
2026-05-14-17-34-34 : jaffa : INFOS : WatchTower started in thread mode
```

FIGURE 17 – Activation du mode Telegram avec l'option `-m` avant supervision d'un scénario lourd.

```
jaffa@Mohammeds-MacBook-Air ~/watchtower-test (main)> ./watchtower -m -t -i 2 -a cpu=20,ram=85,disk=90 -l ./logs -c servers.local.conf
[
2026-05-14-17-35-37 : jaffa : INFOS : Config loaded: 6 server(s)
2026-05-14-17-35-37 : jaffa : INFOS : WatchTower started in thread mode
```

FIGURE 18 – Lancement du programme avec un seuil CPU personnalisé grâce à l'option `-a`.

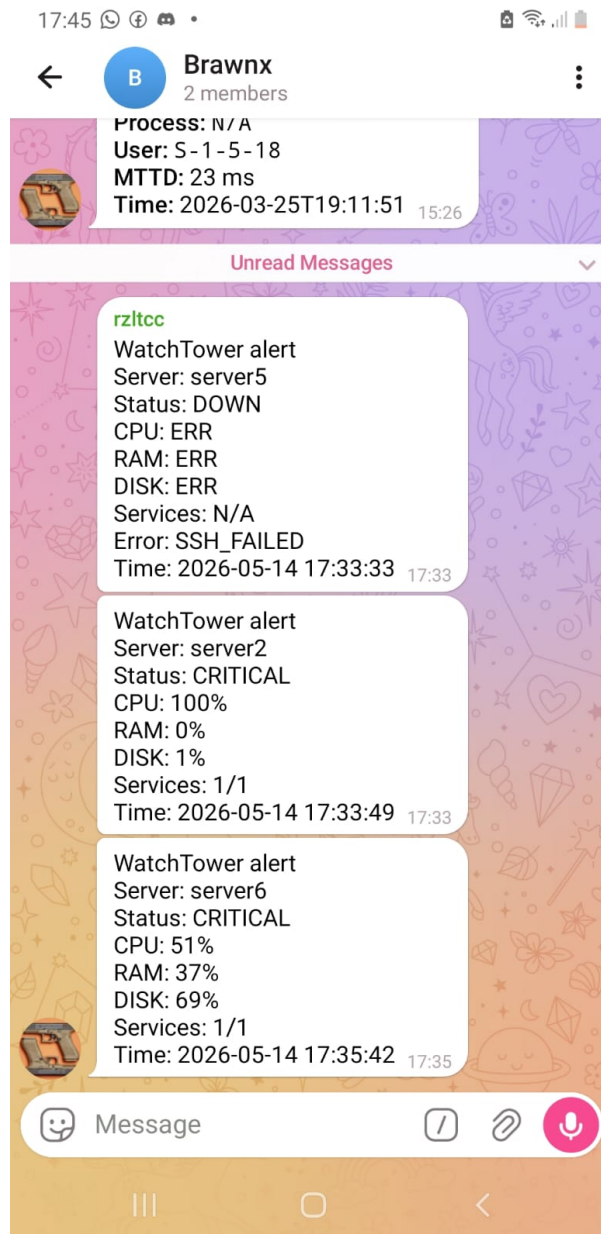


FIGURE 19 – Réception effective des alertes Telegram sur smartphone lorsque les états *DOWN* et *CRITICAL* sont détectés.

13 Analyse des résultats

Les tests réalisés montrent que le projet répond correctement à l'essentiel des consignes du sujet :

- les options obligatoires sont présentes et fonctionnelles ;
- le paramètre obligatoire `-c` est bien pris en charge ;
- le mode `subshell` illustre une exécution séquentielle ;
- le mode `fork` montre clairement la création de processus fils ;
- le mode `thread` s'appuie sur un helper C et journalise les workers créés ;
- les logs sont conservés avec un format structuré ;
- les options de sauvegarde et de restauration sont opérationnelles ;

— les alertes Telegram permettent d’améliorer la réactivité de l’administrateur.

Certaines captures mettent aussi en évidence des cas d’échec réels, par exemple l’indisponibilité d’un serveur Windows ou le dépassement d’un seuil CPU. Cela confirme que le script ne se limite pas à un affichage théorique : il gère effectivement des anomalies d’exécution.

14 Conclusion

Le projet WatchTower répond au besoin d’automatiser la supervision de plusieurs serveurs dans un environnement hétérogène. Le script développé respecte les principales exigences de l’énoncé : utilisation de commandes Unix/Linux, paramètre obligatoire, plusieurs options, gestion des erreurs, journalisation normalisée, test de charges légères, moyennes et lourdes, exécution en `subshell`, `fork` et `thread`, ainsi qu’une documentation intégrée.

L’ajout d’une sauvegarde des journaux, d’une restauration administrateur et d’alertes Telegram renforce encore la pertinence pratique de la solution. Ce mini projet constitue donc une application complète, cohérente et réaliste des concepts étudiés dans le module.